

Arreglos de estructuras

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo



Arreglos de estructuras

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo

Objetivo General

Aplicar arreglos de estructuras para organizar múltiples conjuntos de datos relacionados, permitiendo almacenar y manipular registros de forma estructurada.

Objetivos Específicos

- Declarar y usar arreglos de estructuras para representar listas de objetos (como estudiantes, productos, etc.).
- Crear ciclos para ingresar, procesar y mostrar datos almacenados en arreglos de tipo struct.

¿Qué es un arreglo de estructuras?

Un arreglo de estructuras **es una colección de varias estructuras del mismo tipo**, almacenadas en una sola variable compuesta, como si fueran una lista.

Esto permite tener, por ejemplo, una lista de estudiantes, una tabla de productos, etc.

Sintaxis básica:

```
struct Estudiante {  
    string nombre;  
    float nota;  
};
```

Aquí estamos diciendo: "Quiero guardar los datos de 3 estudiantes".

`Estudiante alumnos[3]; // Arreglo de 3 estructuras tipo Estudiante`

Cuando tienes que almacenar **varios conjuntos de datos del mismo tipo**, pero que incluyen **múltiples atributos**, lo mejor es usar una estructura... y cuando son **muchos registros**, lo ideal es usar un **arreglo de estructuras**.

Los arreglos de estructuras en C++ son comúnmente unidimensionales, pero pueden ser bidimensionales o dinámicos si se usa memoria dinámica. Además, sí se pueden ordenar usando algoritmos como `sort()` u ordenamiento manual, organizándolos por un atributo como nombre o nota.

Ejemplo básico de arreglo de tipo estructura

```
main.cpp
1  #include <iostream>
2  using namespace std;
3
4  struct Estudiante {
5      string nombre;
6      float nota;
7  };
8
9  int main() {
10     Estudiante alumnos[3]; // Arreglo de 3 estudiantes
11
12     // Ingreso de datos
13     for (int i = 0; i < 3; i++) {
14         cout << "\nEstudiante " << i + 1 << endl;
15         cout << "Nombre: ";
16         cin >> alumnos[i].nombre;
17
18         cout << "Nota: ";
19         cin >> alumnos[i].nota;
20     }
21
22     // Mostrar datos
23     cout << "\n--- LISTADO DE ESTUDIANTES ---\n";
24     for (int i = 0; i < 3; i++) {
25         cout << "Nombre: " << alumnos[i].nombre << ", Nota: " << alumnos[i].nota << endl;
26     }
27
28     return 0;
29 }
```

Aquí estamos diciendo: "Quiero guardar los datos de 3 estudiantes".

¿Qué ocurre aquí?
Se declara un arreglo alumnos[3] de tipo Estudiante.

Se usa un for para ingresar los datos de cada uno.

Luego se usa otro for para mostrar todos los datos.

Output

Estudiante 1
Nombre: Novato
Nota: 90

Estudiante 2
Nombre: Novato1
Nota: 60

Estudiante 3
Nombre: Novato2
Nota: 45

--- LISTADO DE ESTUDIANTES ---
Nombre: Novato, Nota: 90
Nombre: Novato1, Nota: 60
Nombre: Novato2, Nota: 45

Ventajas:

- Permite guardar múltiples registros organizados.
- Evita tener que crear una variable por cada persona o producto.
- Facilita la lectura, almacenamiento y procesamiento de información.

Nota: Recuerda que puedes probar el código con un compilador en línea.
<https://www.programiz.com/cpp-programming/online-compiler>

Arreglos de estructuras

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo

Arreglos de estructuras

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo

